GPU-Accelerated Jigsaw Puzzle Classification

GPU Programming and Architectures – SS 2026

S. Dengg K. Hauser J. Polzer J. Janser J. Müller

TU Wien

June 23, 2026

Outline

Motivation & Problem

Pipeline Overview

Image Processing

Boundary Extraction

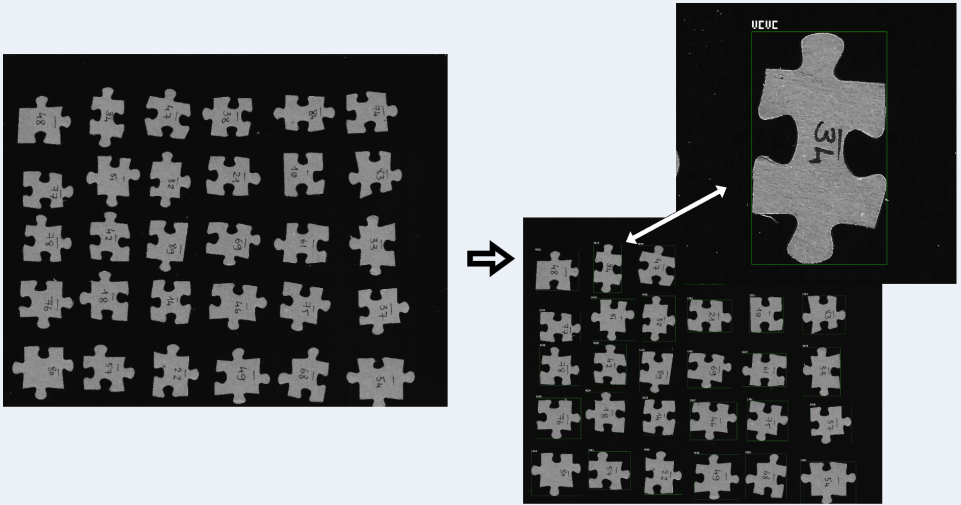
Signal Analysis

Visualization & Optimizations

Results

Conclusion & Future Work

Motivation



Puzzle Piece Classification

The Task

Given a photo of scattered jigsaw pieces, detect every piece and classify each of its **four edges** as one of three types.

L – Straight

V – Knob

C – Hole

Solving the Task on GPU

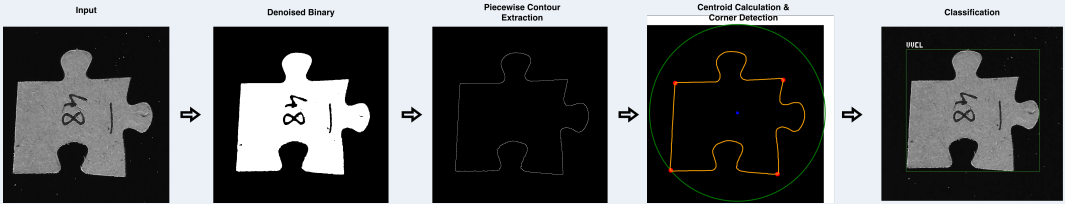
Challenges

- High-resolution images $\rightarrow$ 5100$\times$7016 pixels
- Many independent pieces per image
- Multiple stages: filtering, labeling, signal processing
- Identify parallelizable stages

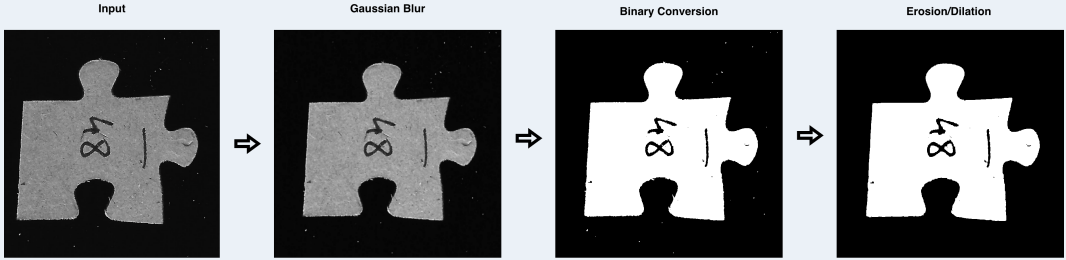
Methodology

End-to-End Flow – Raw JPEG in, annotated JPEG out

1. **Image Preprocessing** – decode, blur, threshold, morphology → binary mask
2. **Boundary Extraction** – CCL, contour ordering, centroid → ordered 1D signal
3. **Signal Analysis & Visualization** – smooth, detect corners, classify, render



Preprocessing Pipeline



Stage 1a – Decode & Grayscale

nvJPEG Decode

- Decode directly into GPU memory – no host→device transfer
- `nvjpegGetImageInfo` reads the header first to size buffers
- `NVJPEG_OUTPUT_RGBI` → interleaved RGB values

RGB → Grayscale

- One thread per pixel
- Conversion to single-channel grayscale array
- Simplifies thresholding for the later stage

Stage 1b – Gaussian Blur

Separable Convolution

Separable Gaussian $G(x, y) = g(x)\, g(y)$, $\rightarrow$ one horizontal and one vertical 1D pass:

$$I'(x, y) = \sum_{j=-r}^{r} g(j) \sum_{i=-r}^{r} g(i)\, I(x + i,\ y + j)$$

Coefficients

- $r = \lceil 3\sigma \rceil$; $\sigma = 2 \Rightarrow$ 13-tap
- Normalized on host, stored in constant memory
- `#pragma unroll` on both convolution loops
- Border pixels: coordinate clamped (replicate)

Fused Single Kernel

- `smem_in[BLOCK_Y + 2r][BLOCK_X + 2r]` - input tile with full halo
- Horizontal pass over **all** BLOCK_Y $+ 2r$ rows $\rightarrow$ `smem_hc`
- Vertical pass reads `smem_hc` - no global memory access

Stage 1c – Histogram & Otsu Thresholding

Histogram (256 bins)

- Vectorized loads - 4 pixels per thread
- Each warp accumulates into a private shared-memory slice via `atomicAdd`
- All warp slices merged in shared memory first → one `atomicAdd` per bin to global `hist[]`

Otsu – Automatic Threshold

- One block, 256 threads – one per bin
- Hillis–Steele prefix scan $O(\log 256)$ builds cumulative count and weighted sum per threshold t
- Each thread computes $\sigma_B^2(t) = \omega_0\,\omega_1(\mu_0 - \mu_1)^2$ independently
- Argmax via shared-memory parallel reduction → threshold

Stage 1d – Binarization

Thresholding

Pixels above the Otsu threshold $\rightarrow$ 255 (foreground), all others $\rightarrow$ 0 (background), producing the binary mask.

Output

- Noisy binary image of pieces

Stage 1e – Erosion & Dilation

Shared Memory Tiling (identical in both kernels)

- Each block loads $(\text{blockDim} + 2r)^2$ tile into `__shared__` memory
- pixels fetched **once** from global memory $\rightarrow$ reused by all overlapping $(2r+1)^2$ kernels.
- Out-of-bounds pixels clamp to 0
- `__syncthreads()` barrier before any thread reads the tile

Erosion

- Output $= 255$ iff **all** neighbours $= 255$
- One non-white neighbour $\Rightarrow$ pixel removed
- Shrinks foreground, kills isolated noise

Dilation

- Output $= 255$ iff **any** neighbour $= 255$
- One white neighbour $\Rightarrow$ pixel added
- Expands foreground, restores piece size

Stage 2a – Connected Component Labeling

Playne-Equivalence Algorithm [1]

- GPU-optimized CCL – avoids heavy global atomics and branch divergence
- Two phases: Local CCL $\rightarrow$ Global Merging
- Output: every pixel of a piece shares one unique label (its root index)

CCL – Local Blocked Phase

Full CCL Within Each Tile

Each thread block runs a complete CCL on its tile; results are merged later.

Neighbor Discovery

- 4-Way Connectivity
- Optimized pixel reads using `__shfl_up_sync`

Union-Find

- Label will be smallest pixel index of a pixel (root) within the component
- **Initialization:** Pixel inherits Top or Left neighbours label (**Top** takes precedence)
- Builds pointer chain towards the root
- **Path compression:** follow pointers to the root
- **Reduction** merges the remaining missidentified clusters

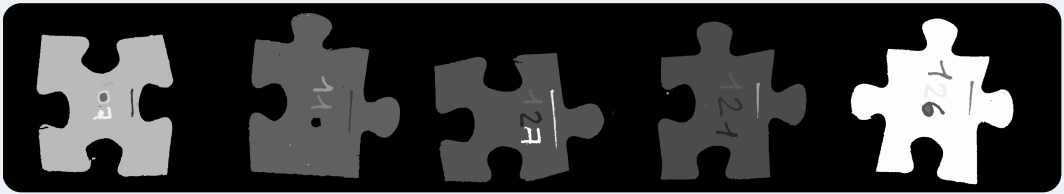
CCL – Border Reduction & Label Propagation

Border Reduction

- Local phase only labels within tiles
- Specialized kernels merge components that **span tile seams**
- Processes horizontal and vertical boundaries

Label Propagation

- Every pixel traverses updated pointers to its final root
- Result: globally unique label per piece
- Label value = global pixel index of the root



Stage 2b – Contour Ordering (List Ranking)

Boundary Detection and List Generation

Boundary pixels (foreground with ≥ 1 background neighbour) are linked into a **clockwise circular list** via Moore-neighbourhood scanning

Parallel List Ranking

- One Thread per Piece traverses the boundary
- Rank (distance from root) is accumulated each step
- Each pixel is assigned a unique rank

Output per Boundary Pixel

- **Label** – which piece it belongs to
- **Rank** – sequential index along the contour
- Packed into `64-bit uint2` for coalesced access

Stage 3a – Distance Signal Generation

Goal

For each puzzle piece, generate a **1D distance signal** from its ordered contour pixels. Each signal value represents the distance of a contour point to the centroid of the corresponding piece.

$$d_i = \sqrt{(x_i - c_x)^2 + (y_i - c_y)^2}$$

Pipeline

- Compact boundary pixel ids
- Create and sort signal points
- Compute centroids from sorted points
- Filter out small components
- Compact valid centroids
- Apply centroids and remove noise

Stage 3a – Preparing Ordered Signal Points

Compact Boundary Pixel IDs

- The labeled image still contains many non-contour pixels
- Implemented with:
  - `thrust::make_counting_iterator`
  - `thrust::copy_if`
- Result: compact array of boundary pixel ids

Create Signal Points

For each contour pixel, a signal point structure is created:

$$p_i = (x_i,\ y_i,\ \texttt{label}_i,\ \texttt{rank}_i,\ d_i)$$

- x_i, y_i: contour pixel position
- `label`: piece/component id
- `rank`: order along the contour
- $d_i = 0$ initially

Stage 3a – Sorting by Piece and Contour Order

64-bit Sorting Key

Signal points are sorted with a combined key:

$$\texttt{key} = (\texttt{label} \ll 32) \mid \texttt{rank}$$

Sorting Effect

- Upper bits contain the component label
- Lower bits contain the contour rank
- Sorting groups all pixels of the same piece together
- Within each piece, pixels are ordered along the contour

Why This Matters

- Enables efficient segmented reductions by label
- Produces signal points in the final signal order
- Avoids an additional reordering step later

Stage 3a – Centroid Computation from Signal Points

Reduction by Label

Each sorted signal point is transformed into:

$$(x_i, y_i, 1)$$

Then a reduction by `label` computes:

$$\left(\sum_{i \in l} x_i, \sum_{i \in l} y_i, n_l \right)$$

for each component label l.

Centroid Formula

The centroid of component l is obtained by:

$$c_x(l) = \frac{1}{n_l} \sum_{i \in l} x_i$$

$$c_y(l) = \frac{1}{n_l} \sum_{i \in l} y_i$$

- One centroid per component

Stage 3a – Noise Filtering and Valid Centroids

Filter Small Components

- Very small components are treated as noise
- A component is valid only if its contour pixel count is large enough:

$$n_l \geq n_{\min}$$

- Filtering is again implemented with `copy_if`
- Invalid labels are ignored in the final signal

Compact Valid Centroids

- Only centroid data of valid components is copied
- The final centroid coordinates are computed during compaction

$$c_l = \left(\frac{\sum x_i}{n_l}, \frac{\sum y_i}{n_l} \right)$$

- Prevents noisy components from affecting later stages

Stage 3a – Applying Centroids to Generate the Signal

Distance Computation

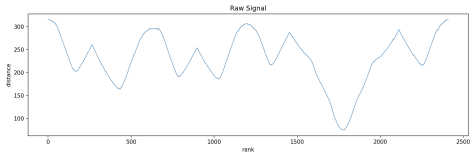
For every valid sorted contour point $p_i = (x_i, y_i)$, the centroid of its component is applied:

$$d_i = \sqrt{(x_i - c_x(\texttt{label}_i))^2 + (y_i - c_y(\texttt{label}_i))^2}$$

Output

- Distances are written to the result array
- Points belonging to noise components are skipped
- The result is already grouped by puzzle piece
- Within each group, the values follow the contour order

Resulting Signal



Stage 3b – Signal Smoothing

Moving-Average Filter

$$D'(i) = \frac{1}{2r+1} \sum_{j=-r}^{r} D(i+j)$$

One thread per boundary pixel; suppresses jitter while preserving corner peaks.

Circular Convolution

- Contours are **closed loops** – window must wrap
- Tail samples correctly feed the start of a segment

Implementation

- `start = tid - rank` locates each piece's base offset
- No per-piece kernel launch – single kernel for all
- `rank`, `segment_length` precomputed on device

Stage 3c – Corner Detection

Two-Stage Approach

A piece has **exactly four corners** – heavy search on GPU, logical selection on CPU.

GPU – Candidate Generation

- Candidate if local maximum within radius R
- Prominence filter: $P = D_i - \frac{1}{2R} \sum_{j \neq 0} D_{i+j} \geq \tau$
- Removes noise-induced micro-peaks

CPU – Best-4 Selection

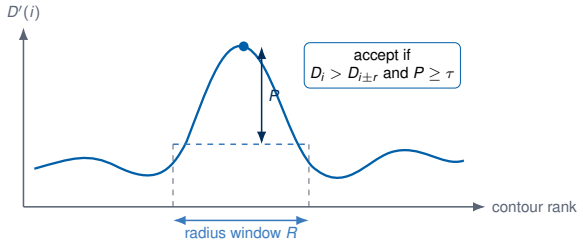
- Sort candidates by prominence
- Accept only if cyclic distance $> L/8$ to existing corners
- Re-sort the final four by contour rank

Stage 3c – Corner Detection: Signal Peaks

Idea

Corners become strong peaks in the smoothed centroid-distance signal. The GPU tests every boundary point as a possible peak in parallel.

Local Maximum + Prominence



GPU Candidate Kernel

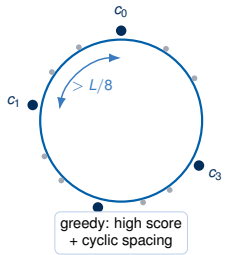
- One thread per smoothed contour point
- Compare against both cyclic neighbours for $r = 1 \ldots R$
- Store `corner_flag[i]` and `score[i]`
- Only flags and scores are copied back for final selection

Stage 3c – Corner Detection: Best Four Corners

Why a Second Stage?

GPU finds many peak candidates. CPU keeps the four strongest **well-separated** corners per piece.

Circular Contour Selection



Selection Pipeline

GPU: flag + score

↓

sort by score

↓

check cyclic distance

↓

keep 4 corners

↓

sort by rank

Stage 3d – Edge Classification

Signed Deviation from Baseline

The four corners split the contour into four edges. For each, the **signed deviation** of contour points from the straight corner-to-corner baseline determines the class (central part only, to avoid corner artifacts).

L – Straight

- Deviation ≈ 0
- Stays on the baseline

V – Knob

- Large **positive** deviation
- Bulges outward

C – Hole

- Large **negative** deviation
- Dips inward

Stage 3d – Edge Classification: Signed Baseline Deviation

Core Idea

Adjacent corners define a baseline. The central half of the edge is compared against this line: outward mean deviation → V, inward mean deviation → C, near zero → L.

One Edge as a Geometry Test

$\bar{d} = \frac{1}{N} \sum d_i$
threshold gives edge class



Signed Distance

$$u = B - A$$

$$s_i = -\frac{u \times (P_i - A)}{\|u\|} \cdot \text{sign}(u \times (C - A))$$

- A, B: neighbouring corners
- P_i: sampled contour point
- C: piece centroid
- $s_i > 0$: outward, $s_i < 0$: inward

Stage 3d – Edge Classification: Decision Rule

Four Corners Split the Ordered Contour into Four Segments

For every segment, compute the mean signed deviation and assign one of three labels.

L – Straight



$|\bar{d}| \leq \tau_e$

V – Knob



$\bar{d} > \tau_e$

C – Hole



$\bar{d} < -\tau_e$

Output per Piece

The result is a 4-character edge string, e.g. LVCC, ordered by contour rank and then rotated in the visualizer so the displayed label starts at the top edge.

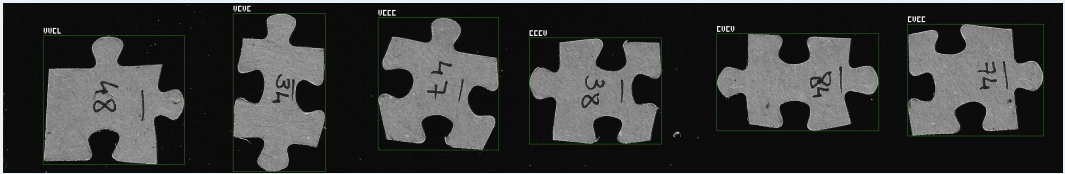
Stage 3e – Visualization

Pixel-Issue Architecture

- Host computes annotation positions (boxes, points, characters)
- Builds a `host_vector` of `issued_pixel_t`
- Single GPU kernel writes all annotations in parallel
- Image never leaves the GPU until encode

nvJPEG Hardware Encode

- CPU encoders require a full image download
- nvJPEG encodes directly from GPU memory
- Only the compressed bitstream goes to the host
- ∼order-of-magnitude latency reduction

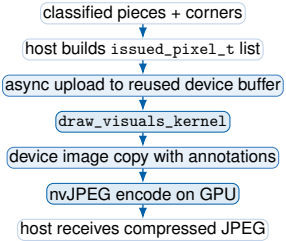


Stage 3e – Visualization: Pixel-Issue Rendering

Goal

Draw bounding boxes, edge labels, optional corner markers, and optional coloured edges without downloading the full raw image to the CPU.

Rendering Dataflow



Why This Is Fast

- Annotation commands are tiny compared to the full image
- The kernel writes independent pixels in parallel
- Raw image stays in GPU memory until encoding
- Workspace buffers and JPEG encoder state are reused across images

Global Optimizations

Memory

- 64-bit packing in pointer jumping – one coalesced transaction per thread
- Shared memory for neighbour lookups (CCL, blur)
- Warp shuffles eliminate redundant global reads
- Small-component pruning shrinks the working set

Transfer

- Data resides on GPU across **all** stages
- nvJPEG decode + encode keep raw pixels on device
- Host receives only the final compressed JPEG
- Pixel-Issue rendering avoids a round-trip
- `cudaStream` for parallelization of multiple images

Benchmarking Results

Execution Times

Stage	Python [s]	GPU [ms]
Preprocessing	140.99	41.985
CCL & Pointer Jumping	677.05	13.998
Signal Generation	–	1.962
Corner Detection	–	0.298
Classification	6.353	0.168
Visualization	20.27	16.219
Total Execution	**844.67** s	**76.106** ms

`18_p2.jpg`, *Resolution:* 5100×7016 *px*
34 pieces, 1 stream, second run

Setup & Observations

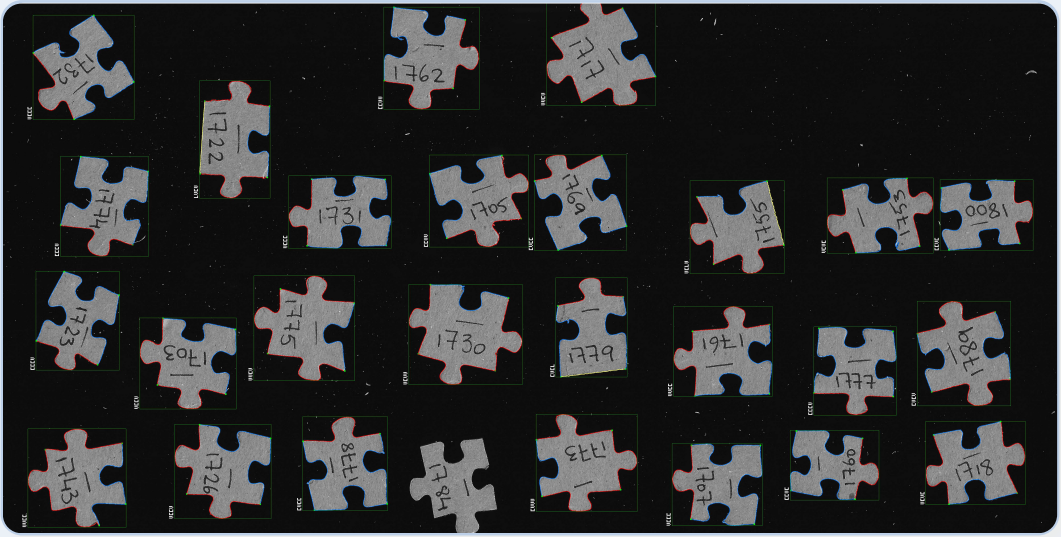
- NVIDIA Tesla T4 (2560 cores)
- AMD EPYC 7313 (∼3 GHz)
- Bottleneck:
  - data transfers
  - JPEG de-/encoding
- Speedup about ×10000
- process all 40 images:
  - 2.2s with 4 streams
  - 3.8s with 1 stream
  - ∼10h in python

Time iterations

Feature	time
primitive implementation	>20s
remove data transfers between functions	10s
reuse jpeg encoder/decoder and memory	4.5s
further code optimizations	3.8s
implement `cudaStreams`	2.1 – 2.3s

time to process all images

Resulting image



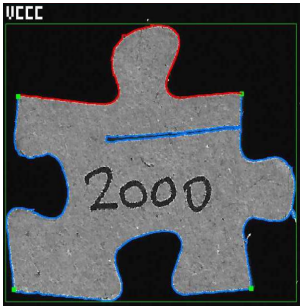
Problems

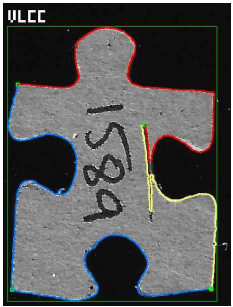
Problem sources

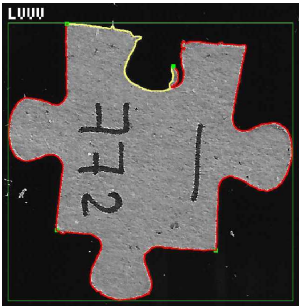
- Dirt on piece edges
- annotation lines over piece edges

Possible problems with different inputs

- different lighting
- light colors
- images on pieces







Possible Improvements

Adaptive Thresholding

- Otsu picks a **global** threshold – fails under strong lighting gradients
- Local / adaptive thresholding would improve robustness
- Fits as a per-tile kernel in the existing pipeline

Filter out problems on pieces

- Filter out lines/text on the edges
- detect text on the piece vs background

Dynamic parameters

classification parameters depending on:

- image resolution
- piece size

Better data

- Images with less dirt
- No drawn lines on the pieces over the border

Conclusion

What We Built

A fully GPU-accelerated pipeline from JPEG input to annotated JPEG output, classifying all four edges of every piece as L, V or C

Key features

- Playne-Equivalence CCL with warp-level optimizations
- Two-stage corner detection (GPU candidates + CPU selection)
- Pixel-Issue rendering + nvJPEG – data never leaves the GPU
- `cudaStream` for multiple image

Outcome

- All stages parallelized; no CPU bottleneck in the hot path
- Raw pixels stay on-device end to end
- Robust, adaptive segmentation (Otsu)
- about $\times$10.000 speedup (vs. python)
- bottleneck data copy from/to GPU

References

[1] D. P. Playne and K. Hawick, “A new algorithm for parallel connected-component labelling on gpus,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 6, pp. 1217–1230, 2018. DOI: 10.1109/TPDS.2018.2799216.